



LICENCIATURA EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA

Interação em Ambientes Virtuais

2º Semestre 2024/2025

Projeto Final

Docente: Engº Arnaldo Abrantes

29/06/2025

Turma: 62D

Trabalho realizado por:
Daniela Santos: A50032
Marta Ferreira: A50775

Conteúdo

Lista de Figuras	2
1 Introdução	3
2 Enquadramento Teórico	4
2.1 Geração de Ambientes Utilizando Perlin Noise	4
2.2 ML-Agents	5
2.3 Interação Multimodal em Jogos	6
3 Ambiente Criado	7
3.1 Mundo Minecraft	7
3.2 Ambiente de Aprendizagem	7
3.2.1 Algoritmo Utilizado	7
3.2.2 Análise das Observações	7
3.3 Sistema de Interação com o Utilizador	8
3.3.1 Comunicação Python-Unity	8
3.3.2 Mapeamento Gestual	8
4 Discussão dos Resultados	9
5 Conclusões	10
6 Bibliografia	11
Referências	11

Lista de Figuras

1	Perlin Noise	4
2	Funcionamento de ML-Agents	5
3	Interação Multimodal	6

1 Introdução

Este projeto final da unidade curricular de Interação em Ambientes Virtuais teve como objetivo a consolidação da matéria aprendida ao longo do semestre, de modo a criar um ambiente virtual em *Unity* que consista numa recapitulação dos conteúdos lecionados.

Neste trabalho, foi desenvolvido um mundo virtual estilo *Minecraft*, elaborado no primeiro trabalho, em que o utilizador poderá controlar o *Player* utilizando os movimentos da mão, captados por uma *webcam*. Por fim, foi adicionado um agente inteligente *ML-Agents* que irá perseguir o utilizador.

O resultado é uma aplicação interativa onde o utilizador pode explorar e interagir com o mundo virtual utilizando gestos da mão captados por uma *webcam*, como caminhar, rodar e destruir blocos. Paralelamente, é seguido por um agente, simulando assim um comportamento autónomo no ambiente. Este projeto combinou técnicas de visão computacional, comunicação em tempo real entre Python e Unity, e métodos de aprendizagem automática aplicados em ambientes tridimensionais.

Este relatório está estruturado da seguinte forma: o Capítulo 2 apresenta o enquadramento teórico, abordando os principais conceitos utilizados, como Perlin Noise, ML-Agents e Interação Multimodal. O Capítulo 3 descreve o ambiente criado, incluindo a construção do mundo estilo *Minecraft*, o sistema de aprendizagem por reforço e a interface gestual. O Capítulo 4 discute os resultados obtidos, avaliando a eficácia do sistema desenvolvido. Por fim, o Capítulo 5 apresenta as conclusões e considerações finais.

2 Enquadramento Teórico

2.1 Geração de Ambientes Utilizando Perlin Noise

A geração procedural de ambientes é uma técnica essencial em áreas como desenvolvimento de jogos, simulações e visualização científica, permitindo a criação de paisagens, texturas e estruturas de forma dinâmica e realista. Um dos algoritmos mais utilizados para esse fim é o Perlin Noise, desenvolvido por Ken Perlin em 1985. Esse método gera ruído pseudoaleatório com propriedades de suavidade e coerência espacial, tornando-o ideal para simular elementos naturais como terrenos, nuvens e texturas orgânicas.

O Perlin Noise é um algoritmo de ruído gradiente que se distingue do ruído puramente aleatório por apresentar continuidade espacial, permitindo a geração de variações suaves em ambientes virtuais. Isso torna-o particularmente útil para a criação de terrenos naturais em jogos ou simulações, ao contrário de outras técnicas que produzem padrões abruptos. A função é geralmente usada em combinação com múltiplas camadas (octaves), escalas e amplitudes (frequência e persistência), o que permite gerar elevações, depressões e padrões realistas que se assemelham a colinas, vales ou montanhas. Além disso, o Perlin Noise pode ser aplicado tridimensionalmente, permitindo construir mundos volumétricos com complexidade realista.

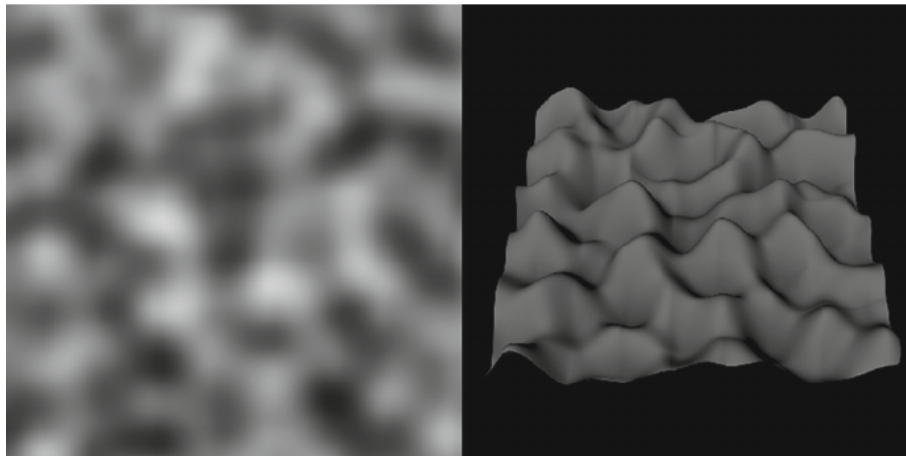


Figura 1: Perlin Noise

2.2 ML-Agents

ML-Agents é uma *toolbox open-source* desenvolvida pela Unity que permite treinar agentes inteligentes em ambientes 3D usando aprendizagem por reforço, aprendizagem por imitação e outros métodos.

A aprendizagem por reforço é uma abordagem em que um agente toma decisões com base nas observações do ambiente, ou seja, observa o estado atual e as recompensas recebidas após cada ação. Ao longo do tempo, o agente ajusta o seu comportamento de forma a maximizar as recompensas acumuladas.

A arquitetura, retratada na Figura 2, envolve o Unity como simulador do ambiente e o Python como interface de treino, através do pacote *mlagents-learn*.

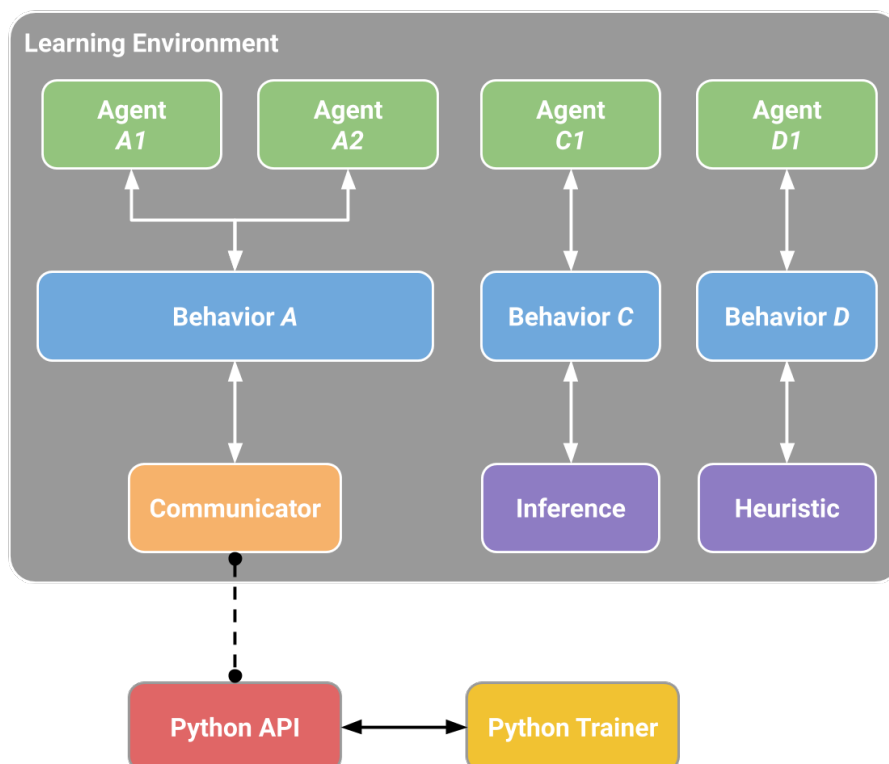


Figura 2: Funcionamento de ML-Agents

2.3 Interação Multimodal em Jogos

A interação multimodal em jogos e ambientes virtuais refere-se à utilização de múltiplos modos de entrada sensorial para comunicar com o sistema. Em vez de depender apenas de teclado ou rato, esta abordagem permite que o utilizador interaja através de gestos, voz, expressões faciais, movimento corporal, entre outros. A combinação destes modos tem como objetivo criar experiências mais naturais, imersivas e intuitivas.

Neste projeto, a interação multimodal é explorada através da visão por computador e reconhecimento de gestos manuais. Utilizando a *webcam* e a biblioteca MediaPipe, conseguiu-se identificar *landmarks* (pontos de referência) na mão do utilizador. Estes dados foram interpretados como comandos visuais, que influenciam diretamente o comportamento dos objetos no mundo virtual.

Esta abordagem permite que o sistema reaja em tempo real a diferentes formas de *input*, como a posição do dedo, a distância entre os dedos, ou a configuração dos mesmos, transformando movimentos naturais em ações dentro do ambiente Unity.

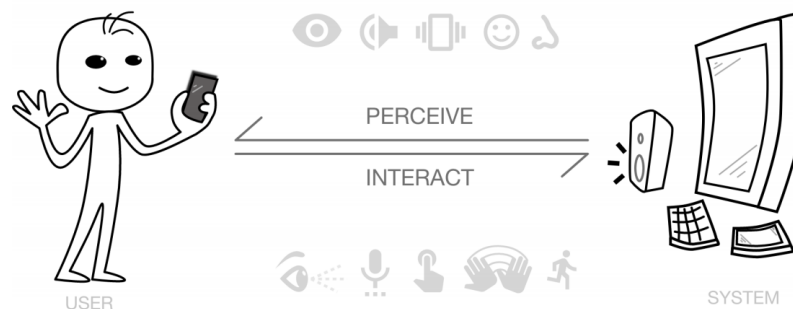


Figura 3: Interação Multimodal

3 Ambiente Criado

3.1 Mundo Minecraft

O ambiente criado simula um mundo semelhante ao Minecraft, construído com blocos cúbicos gerados proceduralmente com recurso ao algoritmo Perlin Noise. Com base nos valores de altura gerados pelo algoritmo, foram colocados blocos de solo para formar relevo e elevações. As regiões abaixo de um determinado nível foram preenchidas com blocos de água, simulando rios ou lagos. Além disso, foram adicionadas árvores simples compostas por blocos de tronco e folhas, posicionadas aleatoriamente em locais com altitude suficiente. Este mundo serve de base para a exploração pelo jogador e para o treino do agente virtual.

3.2 Ambiente de Aprendizagem

3.2.1 Algoritmo Utilizado

Foi utilizado o algoritmo *Proximal Policy Optimization* (PPO), incluído no pacote *ML-Agents* do Unity. Este algoritmo foi escolhido por ser eficaz em tarefas de aprendizagem por reforço em ambientes contínuos ou discretos com alta dimensionalidade.

As ações do agente foram definidas num espaço discreto com três dimensões distintas:

- Movimentação (parado, frente)
- Rotação (nenhuma, direita, esquerda)
- Salto (sim, não)

O agente recebe uma recompensa positiva à medida que se aproxima do jogador e é penalizado por permanecer longe. O mesmo também tem uma pequena penalização por saltar, para evitar que esteja sempre a fazê-lo.

3.2.2 Análise das Observações

Para permitir que o agente tome decisões informadas, foram definidas observações que fornecem uma percepção espacial suficiente sobre a posição do jogador e o estado atual do agente. As observações utilizadas foram as seguintes:

- **Direção para o jogador**

É calculado o vetor normalizado entre a posição atual do agente e a do jogador. Este vetor indica a direção exata que o agente deve seguir para se aproximar do alvo.

- **Distância ao jogador**

A magnitude do vetor entre o agente e o jogador fornece a distância atual entre ambos. Isto permite ao agente saber se está próximo ou longe do objetivo, o que influencia tanto o comportamento de aproximação como a avaliação da recompensa.

- **Direção do agente (forward)**

O vetor `transform.forward` indica a direção atual para a qual o agente está virado. Ao conhecer a sua própria orientação, o agente pode aprender a alinhar-se com a direção do jogador e otimizar os seus movimentos.

Estas observações, combinadas, fornecem uma percepção relativa entre o agente e o jogador, permitindo ao modelo aprender a persegui-lo de forma eficiente, sem o conhecimento absoluto do ambiente.

3.3 Sistema de Interação com o Utilizador

3.3.1 Comunicação Python-Unity

Para permitir que os gestos detetados pela câmara influenciassem o mundo virtual criado no Unity, foi implementado um sistema de comunicação baseado em **UDP (User Datagram Protocol)** entre um *script* em Python e um *script* em C# no Unity. Esta abordagem permitiu uma transmissão rápida e contínua de mensagens contendo os dados dos gestos reconhecidos.

Do lado do Python, a biblioteca MediaPipe, desenvolvida pela Google, foi utilizada para detetar e seguir os pontos-chave (*landmarks*) das mãos em tempo real, a partir da imagem captada pela *webcam*, através da análise da posição relativa dos dedos, o programa identifica quais estão estendidos e interpreta o gesto atual.

Quando um gesto é reconhecido, o Python constrói uma mensagem de texto com a seguinte estrutura:

```
< labeldamao >:< gesto >;< x >;< y >;< valordopintch >
```

Ou seja, por exemplo:

```
right : indicador – polegar;0.3527;0.6271;pintch = 0901
```

Essa mensagem é enviada via UDP para o **IP local (127.0.0.1)** na **porta 9000**, onde o Unity está à escuta.

Do lado do Unity, foi criado um *script* chamado "UDPReceiver", que utiliza uma *thread* em *background* para receber constantemente os dados enviados via UDP. Ao receber uma nova mensagem, esta é encaminhada para o método "InterpretarMensagem"que irá relacionar a mensagem a um dos movimentos se o mesmo estiver associado.

3.3.2 Mapeamento Gestual

Para permitir a interação multimodal com este ambiente, foram feitas associações entre determinados gestos e ações que podem ser realizadas pelo utilizador para controlar o *Player*. A tabela seguinte representa cada função e o seu respetivo gesto.

Mão	Gesto	Ação Associada
Direita	Indicador estendido	Rodar para a direita
Esquerda	Indicador estendido	Rodar para a esquerda
Direita	Dedo do meio estendido	Andar para a frente
Esquerda	Dedo do meio estendido	Andar para trás
Esquerda/Direita	Míndinho estendido	Saltar
Esquerda	Pulgar estendido	Colocar um bloco
Direita	Pulgar estendido	Destruir um bloco

Tabela 1: Mapeamento de Gestos e Ações

4 Discussão dos Resultados

O sistema desenvolvido demonstrou-se eficaz na captação de gestos manuais e na sua tradução em movimentos do jogador dentro do ambiente virtual.

A utilização de comunicação via UDP permitiu uma resposta rápida e com baixa latência, sem perdas de dados significativas durante as sessões de teste. A deteção de dedos individuais mostrou-se estável em movimentos simples e controlados, embora movimentos mais bruscos ou rápidos possam originar falhas no controlo do jogador, afetando a continuidade do movimento.

Em termos de ambiente, o mundo gerado oferece uma base funcional, mas com pouca variedade topográfica, o mesmo poderá ser expandido em termos de ecossistema de forma a criar cenários mais realistas ou desafiantes para o nosso *ML-Agent*.

O agente inteligente mostrou-se funcional na tarefa de seguir o jogador, embora em algumas situações, como ao tentar saltar ou contornar obstáculos, o seu comportamento possa ser otimizado para uma perseguição mais eficiente.

5 Conclusões

O desenvolvimento deste projeto permitiu consolidar conhecimentos fundamentais na área da interação em ambientes virtuais, integrando diferentes tecnologias como geração procedural com Perlin Noise, reconhecimento de gestos com MediaPipe, comunicação em tempo real via UDP e agentes inteligentes treinados com ML-Agents.

A aplicação construída demonstrou ser funcional e interativa, permitindo ao utilizador explorar um ambiente tridimensional de forma natural através de gestos manuais. A resposta aos comandos foi, na maioria dos casos, fluída e precisa, e o agente mostrou comportamento autónomo coerente, mesmo que ainda com margem para melhorias em situações mais complexas.

Este projeto revelou-se um desafio técnico enriquecedor, exigindo a integração de múltiplas áreas como a da visão computacional e inteligência artificial num ambiente de simulação coerente e interativo. Como trabalho final da unidade curricular, representa uma síntese prática eficaz dos conteúdos abordados.

6 Bibliografia

Referências

- [1] Abrantes, A, *Videos de Tutorial no Moodle da cadeira*, 2025.
- [2] Google AI for Developers, *Guia de reconhecimento de gestos para Python*, 2025.
- [3] Unity-Technologies, *Unity ML Agents Toolkit* <https://github.com/Unity-Technologies/ml-agents>, 2025.